Chapter 4: Functions

1. What is a Function?

A function is a named sequence of statements that performs a computation. Think of it as a "sub-program"—a reusable block of code that you "store" and then "reuse" whenever needed.

The Two Types of Functions:

- **Built-in Functions**: These are provided as part of the Python language itself. Examples include print(), input(), type(), float(), and int().
- **User-defined Functions:** These are functions that we create ourselves to solve specific problems.

With parameters	Without parameters

`def function_name(x, y, z):`
`    # instructions`
 	
`    return value`
	`def function_name():`
	`    # instructions`
	
	`    return value`

Pro-Tip: Treat built-in function names as "reserved words." Avoid using names like max or min for your variables, as it can confuse the Python interpreter

2. Built-in Power

Python includes several powerful tools for data manipulation:

- **max() and min():** These find the largest and smallest values in a sequence. For example, max('Hello world') returns 'w'.
- **Type Conversion:** Functions like float() and int() convert data types. Note that int() does not round numbers; it simply "chops off" the decimal part (e.g., int(3.99) becomes 3).
- **Math Module:** By using import math, you gain access to functions like math.sin(), math.log10(), and the constant math.pi

3. Creating Your Own Functions

You define a new function using the def keyword.

The Structure:

1. **Header:** Starts with def, the function name, and parentheses ().
2. **Colon (:):** Every function header must end with a colon.
3. **Body:** The actual code inside the function MUST be indented.
4. **Definition vs. Execution:** Defining a function merely stores the code; it does *not* run it. You must "call" or "invoke" the function by name to actually execute the body.

```
def print_lyrics():
    print("I'm a lumberjack, and I'm okay.")
    print('I sleep all night and I work all day.')
# Calling the function
print_lyrics()
```

4. Arguments and Parameters

Functions are most powerful when you can give them data to work with.

- **Argument:** The actual value you pass *into* the function when calling it (e.g., greet('en')).
- **Parameter:** The variable name used inside the function definition to refer to that incoming data (the "handle").
- **Multiple Inputs:** You can define a function with multiple parameters, separated by commas, such as def addtwo(a, b):.

```
def greet(lang):         # lang is parameter
    if lang == 'es':
        print('Hola')
    elif lang == 'fr':
        print('Bonjour')
    else:
        print('Hello')

greet('fr')             # 'fr' is argument
```

5. Fruitful vs. Void Functions

Functions are categorized by whether or not they "give something back" to the rest of the program.

- **Fruitful Functions:** These perform a calculation and use the return keyword to send a result back. This allows you to assign the result to a variable (e.g., x = addtwo(3, 5)).
- **Void Functions:** These perform an action (like printing text) but do not return a value. If you try to store the result of a void function, you get the special value None.

6. Why Use Functions?

For a Master's student handling complex datasets, functions are essential for several reasons:

- **Organization:** They allow you to organize code into logical "paragraphs" or "chunks".
- **DRY (Don't Repeat Yourself):** Write the code once, debug it once, and reuse it everywhere.
- **Debugging:** Breaking a long program into small functions makes it much easier to find and fix errors one part at a time.
- **Collaboration:** You can build a library of common tools to share with your peers at Univ.